Inteligencia Artificial. Tarea 2

Tomas Santana

Octubre, 2026

1. Descargar un dataset en Kaggle.com (o cualquier otra fuente) con al menos 1000 ejemplos, al menos 8 características predictoras cuantitativas y una variable objetivo continua. El dataset debe ser de datos reales y distinto al utilizado en la Tarea 1. No se permite usar los datasets de los laboratorios (`house_price_regression_dataset.csv` y `diabetes.csv`). Realizar un análisis exploratorio breve y limpiar los datos según sea necesario.

   Separar un conjunto de prueba y escoger un tamaño de entrenamiento lo suficientemente pequeño (o agregar features con `PolynomialFeatures`) para que la regresión lineal **sobreajuste**. Deben mostrar el sobreajuste reportando el RMSE de entrenamiento y de prueba de la clase `LinearRegression` de `sklearn`. (4 puntos)

2. Implementar en Python una clase `MyRidge` que resuelva Ridge regression con la solución cerrada:

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

El intercepto θ_0 no se penaliza. Para ello, la clase debe centrar las features y la etiqueta dentro del método `fit`:

- Calcular la media de cada feature, $\overline{x}_j$, y la media de la etiqueta, $\overline{y}$, usando solamente los datos recibidos en `fit`. Guardarlas como atributos `X_mean_` y `y_mean_`.
- Restar esas medias a X y a y, y resolver la ecuación anterior sobre los datos centrados. La matriz de diseño **no** lleva la columna de unos.
- Recuperar el intercepto como $\theta_0 = \overline{y} - \overline{x}^T\theta$.

En `predict` deben usarse siempre las medias guardadas en `fit`, nunca las de los datos nuevos.

La clase debe recibir el parámetro `alpha` (λ de las notas) en el constructor y tener los siguientes métodos:

- `fit(X, y)`: Ajusta el modelo a los datos de entrenamiento.
- `predict(X)`: Realiza predicciones sobre nuevos datos.
- `score(X, y)`: Calcula el coeficiente de determinación R^2 de las predicciones sobre `X` respecto a `y`, igual que el método `score` de `sklearn`.
- `rmse(X, y)`: Calcula la raíz del error cuadrático medio de las predicciones sobre `X` respecto a `y`.

Como atributos de la clase deben incluirse los parámetros θ (`coef_`), el intercepto θ_0 (`intercept_`) y las medias (`X_mean_`, `y_mean_`), siguiendo la convención de `sklearn`.

Solo pueden usar `numpy`. No pueden usar `sklearn` dentro de la clase. Se recomienda resolver el sistema con `np.linalg.solve` en lugar de calcular la inversa con `np.linalg.inv`, ya que es más estable numéricamente. La clase debe funcionar con features sin estandarizar. (8 puntos)

Ejemplo de uso de la clase:

```
from my_ridge import MyRidge
import pandas as pd

df = pd.read_csv("ruta_al_dataset.csv")
X = df[["feature1", "feature2", "feature3"]].values
```

```
y = df["target"].values

model = MyRidge(alpha=1.0)
model.fit(X, y)

predictions = model.predict(X)
print(model.coef_, model.intercept_)
print(model.score(X, y), model.rmse(X, y))
```

3. Corregir el sobreajuste del punto 1 utilizando su clase `MyRidge` (5 puntos):
   - Estandarizar las features con `StandardScaler`, ajustado **solamente** con el conjunto de entrenamiento.
   - Escoger λ por validación (hold-out o validación cruzada k-fold) sobre una grilla de valores, por ejemplo `np.logspace(-4, 4, 50)`.
   - Graficar el RMSE de entrenamiento y de validación en función de λ, con el eje de λ en escala logarítmica.
   - Reportar el RMSE de prueba del modelo final una sola vez, al final, y compararlo con el de la regresión lineal del punto 1.
   - Entrenar con los mismos datos y el mismo λ la clase `Ridge` de `sklearn`, y verificar con `np.allclose` que `coef_`, `intercept_`, R^2 y RMSE coinciden con los de su implementación. Hacer esta verificación tanto con las features estandarizadas como sin estandarizar.

4. Ridge vs. Lasso (3 puntos). Sobre los mismos datos estandarizados, entrenar las clases `Ridge` y `Lasso` de `sklearn`, escogiendo el λ de cada una por validación cruzada (por ejemplo, con `GridSearchCV` como en el Laboratorio 02).
   - Presentar una tabla con los coeficientes de `MyRidge`, `Ridge` y `Lasso` lado a lado.
   - Graficar los caminos de coeficientes de Ridge y de Lasso en función de λ (escala logarítmica).
   - Reportar cuántos coeficientes de Lasso son exactamente cero y qué features elimina. Comparar el RMSE de prueba de ambos modelos.
   - Explicar por qué Lasso produce coeficientes exactamente iguales a cero y Ridge no, relacionándolo con la geometría vista en clase.

   Importante: `Lasso` de `sklearn` minimiza $\frac{1}{2m}\|y - Xw\|^2 + \alpha\|w\|_1$. El factor $\frac{1}{2m}$ hace que el mismo `alpha` no signifique lo mismo en `Ridge` y en `Lasso`, así que no deben comparar ambos modelos con un mismo valor fijo de `alpha`: cada uno debe escoger el suyo por validación. Con valores pequeños de `alpha`, `Lasso` puede mostrar un `ConvergenceWarning`. En ese caso, aumenten el parámetro `max_iter`.

Entrega y evaluación

Deben entregar un notebook (`.ipynb`) con la solución de todos los puntos y un archivo `my_ridge.py` con la clase `MyRidge`. La entrega es **en clase**. El notebook debe ejecutarse de principio a fin sin errores. La nota definitiva se calculará como:

$$\text{Nota} = \text{Implementación} \times \text{Explicabilidad},$$

donde la implementación es la suma de los puntos anteriores (de 0 a 20) y la explicabilidad es un factor entre 0 y 1 que mide la capacidad de presentar y justificar la solución propia. Para medirla, cada estudiante responderá 3 preguntas sobre su implementación, con su código abierto y ejecutable. El factor se asigna según la siguiente tabla:

Factor	Criterio
1.0	Responde correctamente las 3 preguntas y ubica en su código lo que se le pregunta sin dudar.
0.8	Responde 2 preguntas de forma completa y 1 de forma parcial.
0.6	Responde correctamente 2 preguntas.

0.3	Responde correctamente 1 pregunta.
0	No responde correctamente ninguna pregunta, o no logra ubicar en su propio código lo que se le pregunta.

Una respuesta es **correcta** si contiene todos los puntos esperados y **parcial** si contiene la idea principal pero le falta alguno de ellos.